

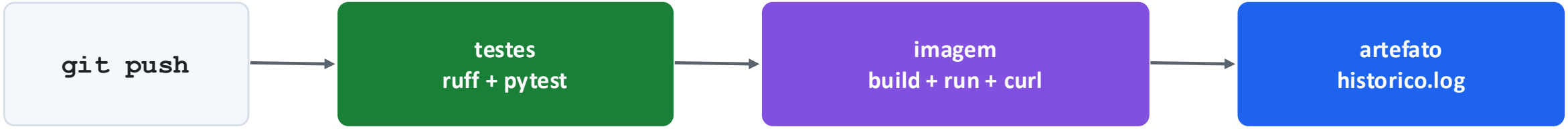
Revisão prática

GitHub Actions + Docker: testar, empacotar e validar uma API no pipeline · 2 horas

DIMAp · Instituto Metrópole Digital · UFRN

O que vamos construir

Uma calculadora web testada no CI, empacotada em imagem e verificada rodando dentro do pipeline — com o histórico das contas guardado em um volume e entregue como artefato.



GitHub Actions Docker Os dois juntos

Dica: Nada de conteúdo novo: workflow, needs, cache, Dockerfile, docker run, volume e artefato — tudo já visto nas aulas.

Roteiro: 6 etapas em 2 horas

1	**Primeiro workflow**	15 min
2	**Testes no CI**	20 min
3	**Dockerfile e execução local**	20 min
4	**A imagem no CI**	20 min
5	**Volume: histórico que sobrevive**	20 min
6	**Histórico como artefato do CI**	20 min

Verde: GitHub Actions · Azul: Docker · Roxo: os dois juntos

O projeto inicial

```
revisao-simples-inicial.zip

revisao-simples/
├── app.py                # a calculadora (Flask)
├── tests/
│   └── test_app.py       # 4 testes
├── requirements.txt      # flask
├── requirements-dev.txt  # + pytest, ruff
├── pyproject.toml        # configuração do pytest
└── README.md
```

Rota	Resposta
`GET /saude`	`{"status": "ok"}`
`GET /soma?a=2&b=3`	`{"a": 2.0, "b": 3.0, "resultado": 5.0}`
`GET /historico`	`["2.0 + 3.0 = 5.0", …]`

Dica: Você só escreve workflows, o Dockerfile e o .dockerignore. O código Python não muda.

Variável HISTORICO

Caminho do arquivo onde cada conta é registrada. Sem ela, a calculadora funciona, mas não guarda nada.

Etapa 1 · Primeiro workflow

15 min | Nível ● ○ ○ ○ ○ | **GitHub Actions** | `workflow_dispatch · inputs · env · runner`

Objetivo Criar o repositório e um workflow manual que recebe um nome e mostra informações do runner.

1. Crie no GitHub o repositório público revisao-simples, clone-o e copie o projeto inicial. Faça commit e push.
2. Crie .github/workflows/ola.yml com workflow_dispatch e um input nome (padrão: turma).
3. Imprima a saudação com o nome — passando o input por env —, o evento e o sistema do runner.
4. Adicione um step com docker --version.
5. Execute pela aba Actions duas vezes, com nomes diferentes.

Verificação

Duas execuções na aba Actions, cada uma com o nome digitado e a versão do Docker do runner.

Desafio extra

Adicione um segundo job que só roda depois do primeiro (needs) e imprime “fim”.

Etapa 1 · Gabarito

```yaml
.github/workflows/ola.yml
name: Olá

on:
  workflow_dispatch:
    inputs:
      nome:
        description: "Quem você quer cumprimentar?"
        default: "turma"

jobs:
  ola:
    runs-on: ubuntu-latest
    steps:
      - name: Saudação
        env:
          NOME: ${{ inputs.nome }}
        run: |
          echo "Olá, $NOME!"
          echo "Evento: $GITHUB_EVENT_NAME"
          echo "Runner: $RUNNER_OS"
      - name: O runner tem Docker
        run: docker --version
```

`inputs.nome`

Aparece como campo de texto no botão Run workflow.

`env: NOME`

Dado digitado pelo usuário chega ao shell por env, nunca interpolado no run.

`$GITHUB_EVENT_NAME`

Variável padrão do runner: workflow_dispatch.

`docker --version`

O runner ubuntu-latest já vem com Docker — vamos usá-lo na etapa 4.

Etapa 2 · Testes no CI

20 min | Nível ●●○○○ | **GitHub Actions** | `push · pull_request · setup-python · cache · pytest`

Objetivo Rodar lint e testes da calculadora automaticamente a cada push na main e a cada pull request.

1. Leia app.py e tests/test_app.py: quais são os 4 testes?
2. Crie .github/workflows/ci.yml disparado por push na main e por pull_request.
3. Job testes: checkout → setup-python 3.12 com cache pip → pip install -r requirements-dev.txt → ruff check . → pytest -v.
4. Faça push e confira o ✓ ao lado do commit.
5. Quebre um teste (espere resultado 6 em vez de 5), veja o ✗ e leia o log. Depois corrija.

Verificação

4 testes passando; o commit quebrado aparece com ✗ e o log mostra a asserção que falhou.

Desafio extra

Transforme o job em matriz com Python 3.12 e 3.13.

Etapa 2 · Gabarito

```yaml
.github/workflows/ci.yml
name: CI

on:
  push:
    branches: [main]
  pull_request:

permissions:
  contents: read

jobs:
  testes:
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-python@v7
        with:
          python-version: "3.12"
          cache: pip
          cache-dependency-path: requirements-dev.txt
      - run: pip install -r requirements-dev.txt
      - run: ruff check .
      - run: pytest -v
```

`push + pull_request`

Valida cada commit na main e cada PR antes do merge.

`permissions`

Menor privilégio: o token só lê o repositório.

`cache: pip`

Instalação mais rápida a partir da 2ª execução.

`ruff antes do pytest`

Erros simples falham primeiro, e mais rápido.

Etapa 3 · Dockerfile e execução local

20 min | Nível ●●○○○ | **Docker** | `slim · ordem das camadas · ENV · USER · run -p`

Objetivo Empacotar a calculadora em uma imagem seguindo as boas práticas e usá-la no seu computador.

1. Dockerfile: python:3.12-slim, requirements.txt antes do código, --no-cache-dir, copie o app.py, ENV HISTORICO=/dados/historico.log, crie a pasta /dados para um usuário não-root, EXPOSE 5000.
2. .dockerignore com .git, .github, tests/, __pycache__/ e .venv/.
3. docker build -t calculadora:1.0 .
4. docker run -d --name calc -p 5000:5000 calculadora:1.0 e teste: curl "http://localhost:5000/soma?a=2&b=3"
5. docker logs calc e docker exec calc whoami.
6. Faça duas contas, rode docker rm -f calc, suba de novo e consulte /historico. O que aconteceu?

Verificação

A soma responde {"a":2.0,"b":3.0,"resultado":5.0}; whoami responde appuser; depois de recriar, /historico volta vazio.

Atenção

Use aspas na URL: o & tem significado especial no terminal. No Windows, use curl.exe.

Etapa 3 · Gabarito

Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
ENV HISTORICO=/dados/historico.log
RUN useradd --create-home appuser && mkdir /dados && chown appuser /dados
USER appuser
EXPOSE 5000
CMD ["python", "app.py"]
```

.dockerignore

```
.git
.github
tests/
__pycache__/
.venv/
```

terminal

```
docker build -t calculadora:1.0 .
docker run -d --name calc -p 5000:5000 calculadora:1.0
curl "http://localhost:5000/soma?a=2&b=3"
docker exec calc whoami    # appuser
```

`requirements.txt` antes

Mudar o app.py não reinstala o Flask (cache).

`ENV HISTORICO`

Dentro do contêiner, o histórico vai para /dados.

`mkdir /dados + chown`

O appuser precisa poder escrever na pasta.

Histórico sumiu?

O arquivo estava na camada de escrita do contêiner removido. A etapa 5 resolve com um volume.

Etapa 4 · A imagem no CI

20 min | Nível ●●●○○ | Actions + Docker | `needs` · `docker build` · `docker run` · `curl` · `failure()`

Objetivo Construir a imagem dentro do pipeline e provar que o contêiner sobe e calcula certo.

1. No ci.yml, crie o job imagem com needs: testes.
2. Construa: docker build -t calculadora:ci .
3. Suba o contêiner em background, com --name calc e -p 5000:5000.
4. Espere a API: um laço que tenta curl -fs /saude até 15 vezes, com sleep 2.
5. Teste a soma: curl -fs "http://localhost:5000/soma?a=2&b=3" | grep '"resultado":5.0'
6. Adicione um step com if: failure() que mostra docker logs calc.

Verificação

O grafo mostra testes → imagem, os dois verdes.

Desafio extra

Quebre o CMD (app2.py): qual job falha e o que os logs mostram? Depois quebre a soma (a - b): quem falha agora, e por que imagem fica skipped?

Etapa 4 · Gabarito

ci.yml — job imagem

```yaml
  imagem:
    needs: testes
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v7
      - name: Construir a imagem
        run: docker build -t calculadora:ci .
      - name: Subir o contêiner
        run: docker run -d --name calc -p 5000:5000 calculadora:ci
      - name: Esperar a API
        run: |
          for i in $(seq 1 15); do
            curl -fs http://localhost:5000/saude && exit 0
            sleep 2
          done
          exit 1
      - name: Testar a soma
        run: curl -fs "http://localhost:5000/soma?a=2&b=3" | grep '"resultado":5.0'
      - name: Logs do contêiner
        if: failure()
        run: docker logs calc
```

`needs: testes`

Só constrói a imagem se o código passou nos testes.

`Laço de espera`

O contêiner leva alguns segundos para começar a responder.

`curl -f + grep`

Falha se a API devolver erro HTTP ou um resultado diferente de 5.0.

`Desafio`

CMD errado: só imagem falha (“can't open file”). Soma errada: testes falha primeiro e imagem nem roda.

Etapa 5 · Volume: histórico que sobrevive

20 min | Nível ●●●○○ | Docker | volume nomeado · -v · docker volume · alpine

Objetivo Guardar o histórico em um volume para que ele sobreviva à remoção do contêiner.

1. docker rm -f calc (da etapa 3) e docker volume create historico.
2. docker run -d --name calc -p 5000:5000 -v historico:/dados calculadora:1.0
3. Faça três somas e consulte /historico.
4. docker rm -f calc, suba de novo com o mesmo -v e consulte /historico.
5. Leia o arquivo sem a API: docker run --rm -v historico:/dados alpine cat /dados/historico.log
6. docker volume ls e docker volume inspect historico.

Verificação

Depois de recriar o contêiner, /historico ainda mostra as três contas; o alpine imprime as mesmas linhas.

Desafio extra

Suba um segundo contêiner na porta 5001 com o mesmo volume e faça uma conta nele. O histórico é compartilhado?

Etapa 5 · Gabarito

```
terminal

docker rm -f calc
docker volume create historico
docker run -d --name calc -p 5000:5000 -v historico:/dados calculadora:1.0

curl "http://localhost:5000/soma?a=2&b=3"
curl "http://localhost:5000/soma?a=10&b=20"
curl "http://localhost:5000/soma?a=1.5&b=2.5"

docker rm -f calc
docker run -d --name calc -p 5000:5000 -v historico:/dados calculadora:1.0
curl http://localhost:5000/historico
# ["2.0 + 3.0 = 5.0","10.0 + 20.0 = 30.0","1.5 + 2.5 = 4.0"]

docker run --rm -v historico:/dados alpine cat /dados/historico.log
docker volume inspect historico
```

`-v historico:/dados`

O volume fica no lugar da pasta /dados do contêiner.

Contêiner descartável

O rm apaga o contêiner; o volume continua.

alpine lendo o volume

Qualquer contêiner pode montar o mesmo volume — até um que não tem a API.

Desafio

Sim: os dois escrevem no mesmo arquivo do volume.

Etapa 6 · Histórico como artefato do CI

20 min | Nível ●●●●○ | **Actions + Docker** | `volume no runner · upload-artifact · etapas 4 + 5`

Objetivo Juntar tudo: o pipeline roda a calculadora com um volume e entrega o histórico como arquivo para download.

1. No job imagem, suba o contêiner com -v historico:/dados.
2. Depois do teste da soma, faça mais duas contas com curl.
3. Exporte o histórico com um contêiner alpine: docker run --rm -v historico:/dados alpine cat /dados/historico.log > historico.log
4. Publique historico.log com actions/upload-artifact (nome: historico).
5. Baixe o artefato na página da execução e confira o conteúdo.

Verificação

O artefato historico aparece na execução e o arquivo tem as três contas, começando por 2.0 + 3.0 = 5.0.

Desafio extra

O step de upload deveria ter if: always()? O que acontece com o artefato se o teste da soma falhar?

Etapa 6 · Gabarito (1/2)

ci.yml — job imagem (início)

```yaml
  imagem:
    needs: testes
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v7
      - name: Construir a imagem
        run: docker build -t calculadora:ci .
      - name: Subir o contêiner com um volume
        run: |
          docker run -d --name calc -p 5000:5000 \
            -v historico:/dados calculadora:ci
      - name: Esperar a API
        run: |
          for i in $(seq 1 15); do
            curl -fs http://localhost:5000/saude && exit 0
            sleep 2
          done
          exit 1
      - name: Testar a soma
        run: curl -fs "http://localhost:5000/soma?a=2&b=3" | grep '"resultado":5.0'
```

Igual à etapa 4…

Build, espera e teste da soma não mudam.

…com `-v historico:/dados`

A única mudança: o mesmo comando da etapa 5, agora no runner.

Etapa 6 · Gabarito (2/2)

ci.yml — job imagem (passos novos)

```yaml
    steps:  # ...continuação
      - name: Mais duas contas
        run: |
          curl -fs "http://localhost:5000/soma?a=10&b=20"
          curl -fs "http://localhost:5000/soma?a=1.5&b=2.5"
      - name: Exportar o histórico do volume
        run: |
          docker run --rm -v historico:/dados alpine \
            cat /dados/historico.log > historico.log
      - name: Publicar como artefato
        uses: actions/upload-artifact@v7
        with:
          name: historico
          path: historico.log
      - name: Logs do contêiner
        if: failure()
        run: docker logs calc
```

Mais duas contas

Para o histórico ter mais de uma linha.

alpine + > historico.log

Outro contêiner lê o volume; a saída vira um arquivo no disco do runner.

upload-artifact

Leva o arquivo para fora da VM, que é descartada ao fim do job.

Desafio

Com if: always() no upload, o histórico é publicado mesmo após uma falha — útil para investigar.

Perguntas de revisão

1. Por que o input do workflow vai para o shell por env?
2. O que o needs: testes garante no job imagem?
3. Por que copiar o requirements.txt antes do app.py no Dockerfile?
4. Imagem × contêiner: qual é imutável? O que some com docker rm?
5. Por que o histórico sumiu na etapa 3 e não na etapa 5?
6. Por que precisamos de upload-artifact para ver o histórico do CI?

Respostas

#	Resposta curta
1	O texto digitado poderia ser interpretado como comando (injeção). Pela env, o shell o trata só como dado.
2	A imagem só é construída se lint e testes passaram; se testes falhar, imagem fica skipped.
3	Cache de camadas: o pip install só é refeito quando as dependências mudam, não a cada mudança no código.
4	A imagem é imutável. O docker rm apaga o contêiner e sua camada de escrita.
5	Na etapa 3 o arquivo estava na camada de escrita do contêiner; na etapa 5, em um volume, que sobrevive ao rm.
6	O runner é uma VM descartável: o que não sai como artefato desaparece ao fim do job.

Checklist de entrega

- [x] ola.yml executado manualmente com um nome (etapa 1)
- [x] ci.yml com os jobs testes e imagem, grafo verde na main
- [x] Dockerfile com slim, ordem das camadas e usuário não-root
- [x] Histórico sobrevivendo à recriação do contêiner (etapa 5)
- [x] Artefato historico na última execução do pipeline

Entregue

Link do repositório e o historico.log baixado do artefato.

Dica: Terminou antes? Adicione a rota /subtracao com teste e verifique-a também no job imagem.



Testado, empacotado e verificado

A cada push, o pipeline testa o código, constrói a imagem, roda o contêiner de verdade e guarda o resultado.

Material: revisao-simples-inicial.zip (projeto inicial) · revisao-simples-gabarito.zip (soluções e o ci.yml de cada etapa)